

LEARNING.md — Blood Report Analyser: Full Onboarding & Architecture Guide

This document is the single source of truth for understanding this project — what it does, how every file works, how AI and MCP fit in, and how to start changing it safely. It was written by reading the entire source code directly (not just the README or the learning journal), so wherever those three disagree, **the code's actual behavior is what's documented here**, with the disagreement called out explicitly.

You do not need prior experience with Python, AI/LLMs, or MCP to follow this document — every term is explained the first time it appears.

Table of Contents

1. [Executive Summary](#)
 2. [What Problem Does This Project Solve?](#)
 3. [Technology Stack](#)
 4. [High-Level Architecture](#)
 5. [Project Directory Structure](#)
 6. [Component-by-Component Explanation](#)
 7. [End-to-End Workflow \(Step by Step, Traced Through the Code\)](#)
 8. [Python Concepts Used in This Project](#)
 9. [AI / LLM Concepts](#)
 10. [MCP \(Model Context Protocol\) Explained](#)
 11. [Application ↔ AI ↔ MCP ↔ External System](#)
 12. [Data Flow](#)
 13. [Configuration & Environment](#)
 14. [Dependencies](#)
 15. [Error Handling](#)
 16. [Logging & Debugging](#)
 17. [Testing](#)
 18. [Running the Application](#)
 19. [Complete Practical Example](#)
 20. [How to Read This Codebase](#)
 21. [Learning Journey & Architectural Decisions](#)
 22. [FAQ — Common Questions a New Developer Will Have](#)
 23. [Code Quality: Strengths, Risks & Improvement Opportunities](#)
 24. [Glossary](#)
 25. [Quick Reference](#)
-

1. Executive Summary

Blood Report Analyser is a small Python backend that takes a photo of a blood test report, reads the numbers off it using an AI model, checks which numbers are outside the normal range, and writes a short, plain-language explanation of what that might mean — always ending with "consult a doctor," never a diagnosis.

It can be used two ways: - As a normal web API (`POST /analyze-report`) that any app can call. - As a **tool an AI assistant can call on its own** (via MCP), so you can literally hand ChatGPT a link to a report image and ask it to analyze it — ChatGPT will call into this exact codebase to do the real work, rather than guessing from the image itself.

There is no database, no user accounts, and no persistent state beyond a few flat files. Everything is deliberately kept simple and visible — this is a learning project (see [Section 21](#)), and the code favors "you can see exactly how this works" over "this uses an industry-standard framework that hides the mechanics."

2. What Problem Does This Project Solve?

Blood test reports are dense and full of jargon (MCV, MCHC, RDW-CV, absolute neutrophil count...). A patient looking at one usually can't tell which numbers matter, or what a "Low" hemoglobin actually implies, without asking a doctor or searching the internet themselves.

This project automates the first pass of that:

- 1. **Read the report** — no manual data entry, just upload a photo.
- 2. **Flag what's abnormal** — compare each value against a known-normal range.
- 3. **Explain it in plain language** — grounded in a small, curated set of health articles, not the AI's own unguided guesses.
- 4. **Always defer to a real doctor** — every response ends with a disclaimer; the system is explicitly designed to never issue a diagnosis or recommend medication.

Who/what interacts with it: - A human, via a web/HTTP client (`curl` , a frontend app, Swagger's UI) calling the REST API directly. - An AI assistant (ChatGPT, confirmed working; any MCP-compatible client in principle) that calls it as a **tool** during a conversation.

External systems it talks to: - **OpenRouter** (`https://openrouter.ai`) — a service that gives access to many different AI models through one API. All AI calls (reading the image, generating advice, creating embeddings) go through OpenRouter, not directly to Google/NVIDIA/etc. - **ngrok** (development only) — creates a temporary public internet address for the app running on your own laptop, so a cloud-hosted AI like ChatGPT can reach it.

Final output of the system, every time: a JSON object with three fields — `biomarkers` (raw extracted numbers), `categorized` (each with a High/Low/Normal tag), and `advice` (the generated explanation text).

3. Technology Stack

Layer	Technology	Where
Language	Python 3.10	entire project
Web framework	FastAPI + Uvicorn (the server that actually runs FastAPI)	main.py

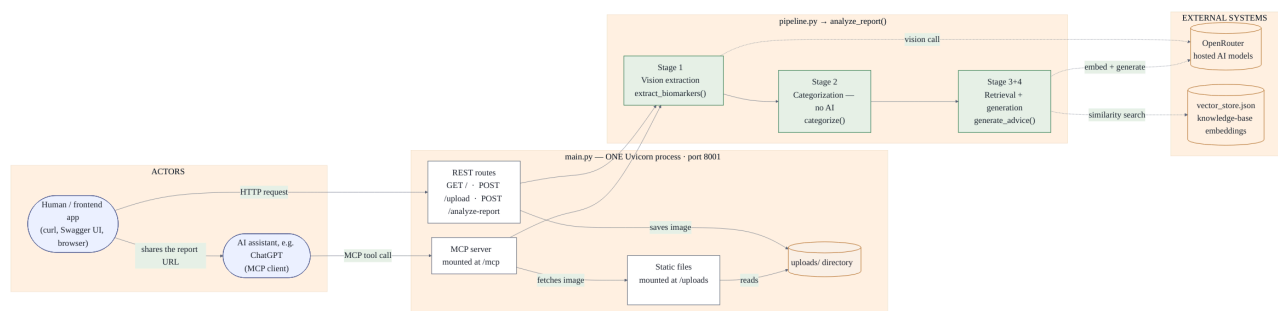
Layer	Technology	Where
AI provider	OpenRouter — a proxy that exposes many AI models through one OpenAI-compatible API	pipeline.py , advisor.py , embedder.py
AI SDK	openai Python package, pointed at OpenRouter instead of OpenAI directly	same three files above
Vision model (reads the image)	<code>google/gemma-4-31b-it:free</code> , with two fallback models	pipeline.py
Text model (writes the advice)	Same three-model list, used for text-only generation	advisor.py
Embedding model (turns text into search-able vectors)	<code>nvidia/llama-nemotron-embed-v1-1b-v2:free</code>	embedder.py
Vector storage	A single JSON file (<code>vector_store.json</code>) — no real database	generated by build_vector_store.py , read by retriever.py
Numeric math	NumPy — used for one thing: comparing two vectors	retriever.py
MCP framework	FastMCP — lets a plain Python function become an AI-callable "tool"	mcp_server/server.py
HTTP client (server-to-server)	httpx — used by the MCP tool to fetch an image from a URL	mcp_server/server.py
Secrets/config	python-dotenv — loads a <code>.env</code> file into environment variables	pipeline.py , advisor.py , embedder.py
Dev tunneling	ngrok — exposes <code>localhost</code> publicly for testing with ChatGPT	not code, a CLI tool run alongside the server

Note on dependencies: there is no `requirements.txt` or `pyproject.toml` in this repo. The only place dependencies are listed is a `pip install ...` line in the README. See [Section 14](#) for the full, verified list (one listed package, `pillow`, turned out to be unused by any code — flagged there).

4. High-Level Architecture

There is exactly **one running process** in this system: a single Uvicorn server running the FastAPI app defined in `main.py`. That one process serves three things at once, all sharing the same port:

- The plain REST API (`/analyze-report` , `/upload`)
- Static file serving for uploaded images (`/uploads/...`)
- The MCP server (mounted at `/mcp`), which is how an AI assistant calls into this app as a tool



► [Mermaid source for the diagram above](#)

The key idea: **both entry points (the REST route and the MCP tool) call the exact same function, `analyze_report()` in `pipeline.py`**. Neither one has its own copy of the pipeline logic. This is intentional — see the "single source of truth" decision in [Section 21](#).

5. Project Directory Structure

```
blood-report-analyser/
├─ main.py                # FastAPI app: HTTP routes + mounts the MCP server
├─ mcp_server/
│   └─ server.py          # The MCP tool: analyze_blood_report(image_url)
├─ pipeline.py            # analyze_report() – orchestrates all 4 stages; also a CLI entry point
├─ categorize.py          # Stage 2: compares values to reference_ranges.py (plain code, no AI)
├─ reference_ranges.py    # The lookup table of "normal" min/max per biomarker
├─ advisor.py             # Stages 3+4: retrieval + AI-generated advice, with model fallback
├─ retriever.py           # Loads vector_store.json, does the similarity search (Stage 3)
├─ embedder.py            # Turns any text into an embedding (a list of numbers)
├─ chunker.py             # Splits long article text into overlapping word chunks
├─ load_articles.py       # Reads every .md file out of articles/
├─ build_vector_store.py  # One-time offline script: articles -> chunks -> embeddings -> vector_st
├─ mock_data.py           # Canned fake results, used when MOCK_AI=true
├─ articles/              # 14 short knowledge-base articles (the RAG source material)
├─ uploads/               # Images saved by POST /upload (not committed to git)
├─ vector_store.json      # Generated embeddings – the actual "database" (not committed to git)
├─ sample_report.png      # A fake/synthetic report image used for manual testing
├─ playground.py         # A scratch file for trying things – not part of the real app
├─ .env                   # Real secrets (API key, etc.) – never committed
├─ .gitignore
├─ README.md              # Project documentation (developer-facing)
└─ LEARNING_JOURNAL.md    # A running diary of what was learned while building this
```

Not documented here (generated/irrelevant): `venv/` (the Python virtual environment — see [Section 8](#) for what that means), `__pycache__/` (Python's compiled-bytecode cache, safe to ignore/delete).

6. Component-by-Component Explanation

For each component: what it is, why it exists, and exactly how it fits with the rest of the app.

6.1 `main.py` — The FastAPI Application (Entry Point #1)

What it is: The web server. This is what you actually start when you run `uvicorn main:app`.

Why it exists: Something has to listen for HTTP requests and translate them into calls to the actual pipeline logic. FastAPI is a Python framework that makes this translation declarative — you write a normal Python function, put a decorator on it, and FastAPI wires it up to a URL.

What it receives: HTTP requests (file uploads, mostly). **What it produces:** JSON HTTP responses. **Who calls it:** Any HTTP client — a browser, `curl`, a frontend app. **What it calls:** `pipeline.analyze_report()` for the real work; it also creates and mounts the MCP sub-application from `mcp_server/server.py`.

Its three routes:

Route	Method	What it does
<code>/</code>	GET	Health check — just returns <code>{"status": "..."} </code> so you can confirm the server is alive.

Route	Method	What it does
<code>/analyze-report</code>	POST	Accepts an image upload, runs the full pipeline , returns <code>{biomarkers, categorized, advice}</code> .
<code>/upload</code>	POST	Accepts an image upload, just saves it to <code>uploads/</code> and returns its URL — no pipeline is run. This exists purely so the MCP tool (see 6.2) has a URL to fetch from.

The most important lines in the file are these, which stitch the MCP server into this same app:

```
mcp_app = mcp_server_instance.http_app(path="/")
app = FastAPI(title="Blood Report Analyser API", lifespan=mcp_app.lifespan)
app.mount("/mcp", mcp_app)
```

- `mcp_server_instance.http_app(path="/")` turns the FastMCP tool server into a small, standalone web app (technically an **ASGI app** — the standard interface Python async web frameworks use to talk to a server like Uvicorn).
- `app.mount("/mcp", mcp_app)` tells FastAPI: "any request starting with `/mcp` should be handed off entirely to this other app instead." This is how one process can serve two logically separate services on one port.
- `lifespan=mcp_app.lifespan` is passed into the **main** FastAPI app, not just the mounted one. This matters because FastMCP needs to run its own startup/shutdown code to manage client sessions correctly — if you forget this, the mounted MCP app looks like it works but session handling breaks in subtle ways. (This exact gotcha is recorded in `LEARNING_JOURNAL.md`.)

6.2 `mcp_server/server.py` — The MCP Tool Server

What it is: A definition of one "tool" — `analyze_blood_report` — that an AI assistant can call.

Why it exists: To let an AI assistant use this app's real logic instead of guessing. Without this file, ChatGPT could only look at a blood report image with its own built-in vision ability and make something up — with this file, ChatGPT can instead call into *this exact, tested pipeline* and get a real, grounded answer back.

What it receives: A tool call from an MCP client, with one argument: `image_url` (a string). **What it produces:** A dictionary — the same `{biomarkers, categorized, advice}` shape as the REST endpoint. **Who calls it:** An MCP client (ChatGPT, in the tested case), through the `/mcp` mount point in `main.py`. **What it calls:** `pipeline.analyze_report()` — the same function the REST endpoint calls.

```

@mcp.tool
def analyze_blood_report(image_url: str) -> dict:
    """
    Analyzes a blood test report image and returns categorized biomarker
    results (High/Low/Normal) along with personalized, non-diagnostic
    health guidance...
    """
    if image_url.startswith("/"):
        image_url = f"{BACKEND_BASE_URL}{image_url}"

    response = httpx.get(image_url, timeout=30)
    response.raise_for_status()
    image_bytes = response.content

    with tempfile.NamedTemporaryFile(delete=False, suffix=".png") as tmp:
        tmp.write(image_bytes)
        tmp_path = tmp.name

    try:
        return analyze_report(tmp_path)
    finally:
        os.remove(tmp_path)

```

Walking through it: 1. `@mcp.tool` is a **decorator** (see [Section 8](#) for what that means) — it's the one line of code that turns a normal Python function into something an AI can discover and call. FastMCP reads the function's name, its type hints (`image_url: str`, `-> dict`), and its **docstring** (the triple-quoted text) to build a description the AI actually reads to decide when/how to use this tool. 2. If `image_url` is a relative path like `/uploads/abc.png` (what `POST /upload` returns), it's turned into a full URL using `BACKEND_BASE_URL`, an environment variable. 3. `httpx.get(image_url, timeout=30)` — the **server itself** fetches the image bytes over HTTP. This is a deliberate design choice explained in 6.2.1 below. 4. `response.raise_for_status()` — if the fetch failed (e.g. 404), this raises an exception immediately rather than silently continuing with garbage data. 5. The bytes are written to a temporary file, because `analyze_report()` (Stage 1 specifically) expects a file path on disk, not raw bytes in memory. 6. `analyze_report(tmp_path)` runs the full 4-stage pipeline — identical to what `/analyze-report` does. 7. `finally: os.remove(tmp_path)` — the temp file is always deleted, whether the analysis succeeded or raised an exception. This is a Python **context-manager-adjacent cleanup pattern**: `try/finally` guarantees the cleanup code runs no matter what.

6.2.1 — Why a URL instead of the image bytes directly?

An earlier version of this tool took `image_base64: str` (the image, base64-text-encoded, passed directly as the argument). This broke in ChatGPT — the client-side step of base64-encoding an 83KB image hung indefinitely and never even reached the server. This is a known limitation across multiple MCP clients with large binary tool arguments. The fix: never send image bytes through a tool argument at all. Instead: - `POST /upload` saves the image and hands back a short URL string. - The AI is only ever asked to pass that short string as `image_url`. - The **server** (not the AI, not the AI's client app) is the one that actually fetches the bytes, using ordinary `httpx.get()`.

This is a standard real-world pattern ("upload to storage, pass a link, let the backend fetch it") — not something invented uniquely for this project.

Note — a previously dead code path was recently removed: this file used to end with:

```

if __name__ == "__main__":
    mcp.run(transport="streamable-http", host="127.0.0.1", port=8765)

```

This let you run `python3 mcp_server/server.py` directly, as a **second, separate server on port 8765**. Once the MCP server was merged into `main.py` (see [Section 21](#)), `main.py` started importing the `mcp` object directly and mounting it — it never runs this file as a script anymore. That block became unreachable dead code (confirmed: nothing in the app calls this file as `__main__`, and port 8765 never appears anywhere else in the codebase) and has since been deleted. If you clone an older commit, you may still see it — it no longer reflects how the app actually runs.

6.3 `pipeline.py` — The Orchestrator (the "brain")

What it is: One function, `analyze_report(image_path) -> dict`, plus the vision-extraction logic that feeds it.

Why it exists: So there is exactly **one** place that defines "what does it mean to analyze a report." Both entry points (6.1 and 6.2) call this, so a bug fix or improvement here automatically applies everywhere.

```
def analyze_report(image_path: str) -> dict:
    biomarkers = extract_biomarkers(image_path) # Stage 1
    categorized = categorize(biomarkers)         # Stage 2

    if MOCK_AI:
        from mock_data import MOCK_ADVICE
        advice = MOCK_ADVICE
    else:
        vector_store = load_vector_store()
        advice = generate_advice(categorized, vector_store) # Stage 3+4

    return {"biomarkers": biomarkers, "categorized": categorized, "advice": advice}
```

This function also doubles as a **CLI entry point** — running `python3 pipeline.py` directly triggers the `if __name__ == "__main__":` block at the bottom, which analyzes the local `sample_report.png` and prints the result to the terminal. This is useful for testing pipeline changes without starting the whole web server.

`extract_biomarkers()` (Stage 1) lives in this file too — see [Section 7](#) for a full line-by-line trace.

6.4 `categorize.py` + `reference_ranges.py` — Stage 2 (Deliberately Not AI)

What it is: A plain function, `categorize(biomarkers) -> dict`, that compares each number against a fixed table of normal ranges.

Why it exists, and why it's *not* an AI call: Comparing a number to a min/max range has exactly one correct answer every time — there's no ambiguity, no judgment call, nothing an AI model would do better than a plain `if` statement. Using AI here would add cost, latency, and a (small but real, and pointless) chance of the model getting a comparison wrong. This is a deliberate, documented architecture decision — see [Section 21](#).

```
if value < ref["min"]:
    status = "Low"
elif value > ref["max"]:
    status = "High"
else:
    status = "Normal"
```

`reference_ranges.py` is just a dictionary — biomarker name → `{"min": ..., "max": ..., "unit": ...}` — for 19 biomarkers (CBC panel values). If a biomarker isn't in this table, it's tagged `"Unknown (no reference range)"` rather than the code guessing.

What it receives: the `biomarkers` dict from Stage 1 (`{"Hemoglobin": 10.6, ...}`). **What it produces:** a richer dict per biomarker: `{"value": 10.6, "status": "Low", "normal_range": "13.0-17.0 g/dL"}` .
Called by: `pipeline.analyze_report()` .

6.5 `advisor.py` — Stages 3 & 4 (Retrieval + Grounded Generation)

What it is: The function `generate_advice(categorized, vector_store) -> str` , plus `call_model_with_fallback()` , the retry/fallback logic shared by every text-generation call.

Why it exists: To turn a list of abnormal values into a human-readable explanation, without the AI inventing medical claims it was never actually given.

Step by step: 1. Filters `categorized` down to only entries where `status` is "High" or "Low" — normal values don't need any explanation. If nothing is abnormal, it returns a canned message immediately, **with no AI call at all**. 2. For each abnormal biomarker, builds a short search query like "Hemoglobin is Low" and calls `retrieve_relevant_chunks()` (from `retriever.py`) to pull back the single (`top_k=1`) most relevant paragraph from the knowledge base. 3. Assembles a prompt containing: a summary of the findings, the retrieved paragraphs (labeled per biomarker), and an explicit instruction: *"Using ONLY the reference information above... Do not diagnose... Always end by recommending they consult a licensed doctor."* 4. Sends that prompt to `call_model_with_fallback()` , which tries a list of AI models in order, retrying each with a growing delay, until one succeeds.

What it receives: `categorized` (from Stage 2) and `vector_store` (loaded from disk by `pipeline.py`). **What it produces:** a string of advice text. **Called by:** `pipeline.analyze_report()` (only when `MOCK_AI` is off). **Calls:** `retriever.retrieve_relevant_chunks()` , then OpenRouter via the `openai` SDK.

6.6 `retriever.py` — Semantic Search (Stage 3's engine)

What it is: Three small functions: `load_vector_store()` , `cosine_similarity()` , `retrieve_relevant_chunks()` .

Why it exists: To answer the question "which paragraphs in our knowledge base are actually *about* this finding?" — without keyword matching (which would miss a paragraph about "anemia" when the query says "Hemoglobin is Low").

```
def cosine_similarity(vec_a: list, vec_b: list) -> float:
    a = np.array(vec_a)
    b = np.array(vec_b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
```

This is the mathematical core of "semantic search." Every piece of text (queries and articles alike) gets converted into a long list of numbers (an **embedding** — see [Section 9](#)) that represents its *meaning*. Two lists of numbers that point in a similar "direction" in that space came from text with a similar meaning. Cosine similarity is just the standard way of measuring "how similar is the direction of vector A to the direction of vector B," returning a number from -1 to 1 (closer to 1 = more similar).

`retrieve_relevant_chunks(query, vector_store, top_k)` : 1. Embeds the query text (calls `embedder.embed_text()` — one more AI/network call). 2. Computes the cosine similarity between that query embedding and **every single chunk's embedding** already stored in `vector_store.json` . 3. Sorts all of them by score, returns the top `top_k` .

This is a **brute-force linear scan** — it checks every chunk, one at a time. At the current scale (a few dozen chunks) this is instant; it would not scale to a knowledge base with millions of chunks without a real vector database (see [Section 23](#)).

6.7 `embedder.py` — Turning Text Into Numbers

What it is: One function, `embed_text(text) -> list`, wrapping a single call to OpenRouter's embedding model.

Why it exists: Both the offline knowledge-base build (6.9) and every live search query (6.6) need to convert text into the same kind of numeric vector, using the exact same model — otherwise the numbers wouldn't be comparable to each other at all. Centralizing this in one function guarantees that.

Includes its own retry loop (3 attempts, waiting longer between each) since embedding calls hit the same free-tier rate limits as any other AI call.

6.8 `chunker.py` + `load_articles.py` — Preparing the Knowledge Base

`load_articles.py` just reads every `.md` file out of the `articles/` folder into a dictionary of `{filename: text}`.

`chunker.py` splits a long piece of text into smaller, overlapping pieces:

```
def chunk_text(text: str, chunk_size: int = 100, overlap: int = 20) -> list:
    words = text.split()
    ...
    start = end - overlap # step forward, but overlap with the previous chunk
```

Why overlap matters: if a chunk boundary fell in the middle of an important sentence, the second half of that sentence's meaning could get lost, disconnected from its first half. Repeating the last `overlap` words at the start of the next chunk keeps sentences that straddle a boundary intact in at least one chunk.

6.9 `build_vector_store.py` — The One-Time Offline Build Script

What it is: A script you run manually (`python3 build_vector_store.py`), not something the running app calls automatically.

Why it exists: Turning 14 articles into a searchable vector store takes real AI calls (one embedding call per chunk) and real time. Doing this once, offline, and saving the result to a file (`vector_store.json`) means every actual user request doesn't have to re-embed the whole knowledge base — it just reads the pre-built file.

It chains together everything from 6.7 and 6.8: `load_articles()` → `chunk_text()` (60-word chunks, 15-word overlap) → `embed_text()` for each chunk → writes the whole list of `{article, chunk_index, text, embedding}` dictionaries to `vector_store.json`. A `time.sleep(1)` between each embedding call keeps the script under OpenRouter's free-tier rate limit.

Must be re-run any time an article in `articles/` is added or edited — it's a full rebuild, not incremental.

6.10 `mock_data.py` — Mock Mode's Fake Data

Just two hardcoded values: `MOCK_BIOMARKERS` (a dict of fake extracted values) and `MOCK_ADVICE` (a fixed advice string). Used only when the `MOCK_AI` environment variable is `"true"` — see [Section 13](#).

6.11 playground.py — Not Part of the Real App

A small, explicitly-labeled scratch file used to manually try things out (currently: running a retrieval query and printing the results). Nothing else in the codebase imports it, and it's not run automatically by anything. It exists so that experimenting doesn't require adding throwaway test code directly into real modules like `retriever.py`.

7. End-to-End Workflow (Step by Step, Traced Through the Code)

There are two possible starting points — a direct REST call, or an AI assistant using the MCP tool. Both converge on the exact same pipeline. This section traces both, in full.

Workflow A — Direct REST API call (`POST /analyze-report`)

```
A1 (client uploads image) → A2 (FastAPI route) → A3 (Stage 1: vision extraction)
→ A4 (Stage 2: categorization) → A5 (Stage 3+4: retrieval + advice) → A6 (JSON response)
```

A1 — Trigger - Input: An HTTP POST request to `/analyze-report`, with the image file attached as `multipart/form-data`. - **Who sends it:** Any HTTP client (browser, `curl`, a frontend app).

A2 — Route handling - File / function: `main.py` → `analyze_report_endpoint(file: UploadFile = File(...))` - **What happens:** FastAPI parses the multipart upload into an `UploadFile` object. The code checks `file.content_type.startswith("image/")` — if not, it immediately raises `HTTPException(400, ...)` and nothing else runs. Otherwise, the uploaded bytes are copied into a temporary file on disk (`tempfile.NamedTemporaryFile`), because the pipeline function expects a file *path*, not raw bytes in memory. - **Calls:** `pipeline.analyze_report(tmp_path)`. - **Output:** whatever `analyze_report()` returns, or an `HTTPException(500, ...)` if it raised an exception. - **Cleanup:** the temp file is deleted in a `finally` block — this runs whether the call above succeeded or failed.

A3 — Stage 1: Vision extraction - File / function: `pipeline.py` → `extract_biomarkers(image_path)` - **Input:** the file path to the image. - **What happens:** the image bytes are read and base64-encoded into a text string, then wrapped in a **multimodal message** — a chat message whose `content` is a *list* with two parts: a text instruction and an `image_url` part containing `data:image/png;base64,<the encoded image>`. This whole thing is sent to OpenRouter, targeting `google/gemma-4-31b-it:free` first. - **The prompt** explicitly demands: *"Respond with ONLY a JSON object, no other text..."* plus one concrete format example. This is **prompt engineering** — see [Section 9](#). - **Retry/fallback:** wrapped in nested loops — for each model in `VISION_MODELS` (3 candidates), try up to `max_retries_per_model` times. If a model's response is empty/malformed, or the call raises any exception, that attempt is recorded as `last_error` and the loop moves on. If every model/attempt fails, the original exception is re-raised. - **Parsing:** `json.loads(raw_output)` turns the model's text response into a real Python dictionary. There's no `try/except` around this specific line inside `extract_biomarkers` — if `json.loads` fails (the model didn't obey the format), that `json.JSONDecodeError` is caught by the broader `except Exception as e:` in the same retry loop and treated like any other failed attempt. - **Mock mode short-circuit:** if `MOCK_AI` is `"true"`, none of the above runs at all — the function returns `mock_data.MOCK_BIOMARKERS` immediately. - **Output:** a dict like `{"Hemoglobin": 10.6, "Platelet Count": 128, ...}`. - **Next:** passed straight into Stage 2.

A4 — Stage 2: Categorization - File / function: `categorize.py` → `categorize(biomarkers)` - **Input:** the dict from A3. - **What happens:** plain Python, no AI call — each value is compared against `REFERENCE_RANGES` from [reference_ranges.py](#) and tagged `High / Low / Normal / Unknown` (no reference range). - **Output:** `{"Hemoglobin": {"value": 10.6, "status": "Low", "normal_range": "13.0-17.0 g/dL"}, ...}`. - **Next:** passed into Stage 3+4 (or skipped in favor of mock advice).

A5 — Stage 3+4: Retrieval + Grounded Generation - **File / function:** `pipeline.py` loads the vector store via `retriever.load_vector_store()`, then calls `advisor.py`'s `generate_advice(categorized, vector_store)`. - **What happens:** described in full in 6.5 and 6.6 above. In short: abnormal findings → search query per finding → top matching knowledge-base paragraph per finding → one big prompt → sent to a text model with the same retry/fallback pattern as Stage 1. - **Mock mode short-circuit:** if `MOCK_AI` is `"true"`, this entire stage is skipped and `mock_data.MOCK_ADVICE` is used instead. - **Output:** a string of advice text.

A6 — Response - **File / function:** back in `main.py`, `analyze_report_endpoint` returns the combined dict `{"biomarkers": ..., "categorized": ..., "advice": ...}`. - **What happens:** FastAPI automatically serializes this Python dict into a JSON HTTP response body with a `200 OK` status. - **Received by:** whatever client made the original request (A1).

Workflow B — An AI assistant uses the MCP tool (e.g. ChatGPT)

```
B1 (upload image via /upload) → B2 (ask ChatGPT to analyze it)
→ B3 (ChatGPT decides to call the tool) → B4 (MCP request reaches the server)
→ B5 (tool fetches the image itself) → B6 (same pipeline as A3-A5 runs)
→ B7 (tool result returned to ChatGPT) → B8 (ChatGPT turns it into a reply)
```

B1 — Get a public image URL - A client (a human, using `curl` or similar, through the ngrok tunnel) calls `POST /upload` with the image. - **File / function:** `main.py` → `upload_report(file)`. Same content-type check as A2, but instead of running the pipeline, it just saves the file under a random name (`uuid.uuid4()`) into `uploads/` and returns `{"url": "/uploads/<uuid>.png"}`. - Combined with the ngrok public address, this becomes something like `https://<ngrok-subdomain>.ngrok-free.dev/uploads/<uuid>.png` — a URL anyone on the internet (including ChatGPT's servers) can fetch.

B2 — The human asks a question in ChatGPT - The user pastes that URL into a ChatGPT conversation and asks it to analyze the report.

B3 — The AI decides to use the tool - This is the one step that happens entirely **inside the AI model**, not in this codebase. ChatGPT reads the available tools it's connected to (via its MCP connector configuration), sees `analyze_blood_report`'s name/description/parameters (auto-generated by FastMCP from the function's docstring and type hints — see 6.2), and decides this request matches that tool. It formats a tool call: `analyze_blood_report(image_url="https://.../uploads/...png")`. - **Important distinction:** this app does not "prompt" ChatGPT or control its reasoning in any way. The AI's decision to call the tool, and how it phrases the final answer, are entirely the AI provider's (OpenAI's) behavior — completely outside this codebase.

B4 — The MCP request reaches the server - ChatGPT's servers send this tool call over the internet to the ngrok public URL, `.../mcp`. - ngrok forwards it to `localhost:8001/mcp` — which is being served by the same Uvicorn process as everything else, at the mount point set up in `main.py` (see 6.1). - FastMCP's internal machinery routes the request to the matching Python function: `analyze_blood_report` in `mcp_server/server.py`.

B5 — The tool fetches the image itself - Exactly as walked through in 6.2: `httpx.get(image_url, timeout=30)` — **the server**, not ChatGPT, performs this fetch. ChatGPT only ever handled a short URL string.

B6 — The exact same pipeline runs - `analyze_report(tmp_path)` is called — this is the identical function used in Workflow A, step A2. Everything in A3, A4, A5 happens exactly the same way. There is no separate "AI path" through the pipeline logic — Stage 1 still calls the vision model, Stage 2 is still plain code, Stage 3+4 still does retrieval + generation the same way.

B7 — The result goes back through MCP - The dict `{"biomarkers": ..., "categorized": ..., "advice": ...}` is returned by the Python function. FastMCP serializes this as the tool's result and sends it back over the

same `/mcp` connection, through ngrok, to ChatGPT's servers.

B8 — ChatGPT turns the raw result into a conversational answer - Again, entirely inside the AI model, not this codebase: ChatGPT reads the structured tool result and writes a human-friendly summary (a markdown table of abnormal results, bullet points for normal ones, a short "what this may indicate" explanation, and a closing doctor-consult recommendation). This was verified working end-to-end — the actual result included a correctly-flagged note that the data was `MOCK_AI` test-mode data, because `MOCK_AI=true` was set on the server at the time.

8. Python Concepts Used in This Project

Only concepts actually present in this codebase are explained here.

def (functions) — every reusable piece of logic in this project is a plain function (e.g. `def categorize(biomarkers: dict) -> dict:`). There are **no classes anywhere in this codebase** — no `class` keyword appears in any file. This is a fully functional, procedural style: data is passed in, transformed, and returned, with no objects holding onto internal state between calls.

Type hints (`image_path: str, -> dict`) — these are annotations telling a reader (and tools like FastMCP) what type of value a function expects and returns. Python does not enforce these at runtime by itself — they're for clarity, and FastMCP specifically reads them to build the MCP tool's schema (so an AI client knows `image_url` must be a string).

Dictionaries (`{}`) — the primary data structure in this project. Biomarkers, categorized results, reference ranges, and tool results are all plain dicts, not custom classes. E.g. `REFERENCE_RANGES["Hemoglobin"]` → `{"min": 13.0, "max": 17.0, "unit": "g/dL"}`.

Dict comprehension — one example, in [advisor.py](#):

```
abnormal = {
    name: info for name, info in categorized.items()
    if info["status"] in ("High", "Low")}
}
```

This builds a new dictionary in one line: "for every `(name, info)` pair in `categorized`, keep it only if the status is High or Low." It's equivalent to a `for` loop that builds up a dict, just more compact.

Generator expression — also in `advisor.py`:

```
findings_summary = "\n".join(
    f"- {name}: {info['value']} ({info['status']}, normal range: {info['normal_range']})"
    for name, info in abnormal.items()
)
```

The part after `"\n".join(` is a generator — it produces one formatted line per abnormal finding, on demand, and `.join()` stitches them together with newlines between them.

lambda — a tiny, unnamed, inline function, used once in [retriever.py](#):

```
scored_chunks.sort(key=lambda x: x["score"], reverse=True)
```

This tells `.sort()` "for each item `x`, sort using `x["score"]` as the comparison value" — without needing to write a separate named function just for that one line.

with (context managers) — used everywhere a file or temp file is opened: `with open(path, "r") as f: ...`, `with tempfile.NamedTemporaryFile(...) as tmp: ...`. `with` guarantees the file gets closed automatically when the block ends, even if an exception happens inside it — you don't have to remember to call `.close()` yourself.

try / except / finally / raise — used for every network call in this project (OpenRouter calls, the image fetch in the MCP tool). The pattern is consistently: try the risky operation, catch failures, sometimes retry, and if all retries are exhausted, `raise` the last error so it isn't silently swallowed. `finally` blocks are used specifically for cleanup that must always happen (deleting a temp file) regardless of success or failure.

async def / await — used in `main.py` for the two FastAPI route functions (`analyze_report_endpoint`, `upload_report`). `async def` marks a function as one that *can* pause and let other work happen while it waits on something slow (like a network call), instead of freezing the whole program. **Notably, neither function actually contains an `await` anywhere in its body** — every call inside them (`shutil.copyfileobj`, `pipeline.analyze_report`, `os.remove`) is a normal, blocking function call. FastAPI still requires/allows `async def` for route handlers as a matter of convention with its underlying server (Uvicorn), even when the handler's own code happens to be fully synchronous. If this project later wanted truly non-blocking I/O here (e.g. calling OpenRouter with an `async` HTTP client), those calls would need `await` in front of them.

Environment variables (`os.getenv("OPENROUTER_API_KEY")`) — a way to pass configuration/secrets into a program from *outside* the code, so secrets never have to be hardcoded or committed to git. `python-dotenv`'s `load_dotenv()` reads a `.env` file and copies its contents into the environment `os.getenv()` reads from. See [Section 13](#).

Imports & modules — every `.py` file in this project is a **module**; `import / from ... import ...` pulls functions from one module into another (e.g. `from categorize import categorize`). `mcp_server/server.py` needs one extra line to make this work across a subfolder boundary:

```
sys.path.insert(0, os.path.join(os.path.dirname(__file__), ".."))
```

This tells Python "also look one directory up when resolving imports," since `pipeline.py` lives in the project root but `server.py` lives inside `mcp_server/`. There is no `__init__.py` file in `mcp_server/` — this project doesn't use it as a formally installable Python package, just as a folder Python can still import a module from directly.

Decorators (`@app.get("/")`, `@mcp.tool`) — a decorator is a function that wraps another function to add behavior, without changing the wrapped function's own code. `@app.get("/")` tells FastAPI "call this function whenever a GET request hits `/`." `@mcp.tool` tells FastMCP "expose this function as an AI-callable tool." Both are examples of the same general pattern: attaching metadata/behavior to a function declaratively.

Virtual environments (venv) — not a language feature exactly, but foundational: a `venv` is an isolated copy of Python plus its own separate set of installed packages, so this project's dependencies don't collide with any other Python project's dependencies on the same machine. You must "activate" it (`source venv/bin/activate`) in every new terminal session before running the app — a very common beginner trip-up, and one specifically called out multiple times in `LEARNING_JOURNAL.md`.

Not used in this project (called out so you don't go looking for them): classes, inheritance, Pydantic models (FastAPI route responses here are plain dicts, not typed response models), dataclasses, `*args / **kwargs`, abstract base classes/ `Protocol`, threading/multiprocessing.

9. AI / LLM Concepts

AI / LLM (Large Language Model) — a model trained on huge amounts of text (and, for some models, images) that can generate text or read images and respond in natural language. This project doesn't train any model — it only *calls* pre-trained models hosted by other companies, through OpenRouter.

Vision-language model (VLM) — an LLM that can accept an image as part of its input, not just text. Used in Stage 1 to literally "read" the blood report photo. In code, this looks like a chat message whose `content` is a list containing both a text block and an `image_url` block, in a single request — this is called a **multimodal** message.

Prompt — the actual text instruction sent to the model. This project uses only **user prompts** — no "system prompt" role is set anywhere in the code (every OpenRouter call uses `{"role": "user", "content": ...}` only).

Prompt engineering — carefully wording the instruction to get a reliable, specific result. The clearest example is Stage 1's extraction prompt, which spells out the exact output format and forbids markdown/prose, specifically because free-tier models don't reliably support a stricter, built-in "JSON mode."

Structured output (via prompting, not a native API feature) — the project asks the model, in plain English, to reply with *only* JSON, then parses that reply with `json.loads()`. This is different from (and less reliable than) a model provider's official structured-output or function-calling feature — the model here has no guarantee it will actually comply, which is exactly why the retry/fallback logic exists.

Tokens — the units LLMs actually process text in (roughly, word-pieces). Not directly manipulated anywhere in this code — the project doesn't count or limit tokens itself — but it's the underlying reason chunking (below) matters at larger scale, and it's what OpenRouter/providers bill by (irrelevant here since only free-tier models are used).

Embeddings — converting a piece of text into a fixed-length list of numbers (a vector) that represents its meaning. `embedder.py`'s entire job is this one conversion. Two texts with similar meaning end up with numerically similar vectors — this is the mechanism that lets `retriever.py` find "the paragraph about anemia" when the query says "Hemoglobin is Low," despite sharing almost no exact words.

Vector store — where those embeddings are kept so they can be searched later. Here, it's the single file `vector_store.json` — a flat list of `{article, chunk_index, text, embedding}` records. This project deliberately does **not** use a real vector database (like Pinecone or Chroma) — the tradeoff is explicit and documented: full transparency into the mechanics, at the cost of not scaling past a small knowledge base.

Chunking — splitting a document into smaller, overlapping pieces before embedding each one (see 6.8). The project's own documentation is candid that this is somewhat unnecessary at its current scale (14 short, single-topic articles) and was implemented mainly for hands-on learning.

Retrieval — the act of searching the vector store for the pieces of text most relevant to a query, using cosine similarity (see 6.6).

RAG (Retrieval-Augmented Generation) — the overall pattern combining the previous two ideas with generation: search your own knowledge base, hand the AI only the relevant retrieved pieces as context, and ask it to answer *using only that context*. This is Stages 3+4 together, implemented in `advisor.py` + `retriever.py`.

Grounding / hallucination mitigation — "hallucination" is when a model states something confidently that isn't actually true or supported by what it was given. This project's generation prompt explicitly says *"Using ONLY the reference information above"* and forbids diagnosis/medication advice — this is the grounding mechanism. A concrete, observed example (from `LEARNING_JOURNAL.md`): when a biomarker was abnormal but had no matching knowledge-base article, the model gave only generic guidance rather than inventing specific-sounding detail it was never given — evidence the grounding is actually working, not just present in the prompt.

Model fallback / resilience — both AI call sites iterate over an ordered list of models (`VISION_MODELS` , `TEXT_MODELS`), retrying each with a growing delay before moving to the next. This exists because free-tier AI models are frequently rate-limited or briefly unavailable, and a single model's hiccup shouldn't fail the whole request.

Not used in this project (worth naming explicitly so you don't assume otherwise): **function/tool calling** at the OpenRouter-call level (the vision/text models here are never given "tools" of their own — only ChatGPT, as an external MCP client, does tool calling, and that happens entirely outside this codebase), **streaming** responses (every AI call here waits for the full response before returning), **conversation memory/history** (every call is a single, stateless, one-shot request — nothing is remembered between calls), and **temperature/other sampling parameters** (none are set explicitly anywhere in the code — each call uses whatever default the model/provider applies).

10. MCP (Model Context Protocol) Explained

What MCP is, in plain terms: a standard way for an AI application (like ChatGPT) to call real code and get real data back, instead of only generating text from what it already knows. Think of it like a universal plug: any AI host that "speaks MCP" can connect to any MCP server and immediately understand what tools it offers, without custom one-off integration code for each pairing.

The problem it solves: without MCP, if you wanted ChatGPT to be able to "run your app," you'd need a custom plugin/integration built specifically for ChatGPT's own plugin system, and a different one again for Claude, and another for the next AI host. MCP is a shared protocol so one server implementation can, in principle, work with any compliant client.

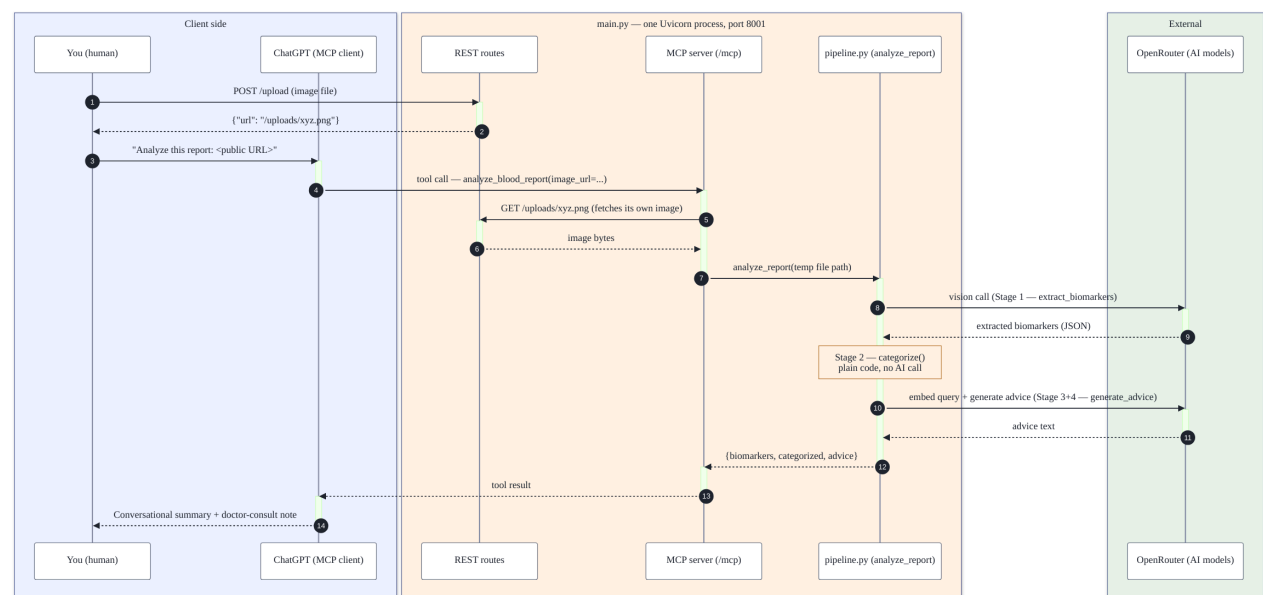
A simple analogy: think of the MCP server as a receptionist at a company. The AI (the visitor) says "I need X." The receptionist (the MCP server) knows exactly which internal department (tool/function) actually does X, goes and gets the real answer, and hands it back — the visitor never needs to know how that internal department actually works.

Key MCP vocabulary, and where each concept lives in this project:

MCP concept	What it means	In this project
MCP server	The program that exposes tools and handles incoming calls	<code>mcp_server/server.py</code> (the <code>FastMCP("Blood Report Analyser")</code> instance), mounted into <code>main.py</code>
MCP client	The AI host application that connects to a server and calls its tools	ChatGPT, configured as a "connector" pointing at this server's public URL
MCP tool	One specific, callable capability, with a name, description, and typed inputs/outputs	<code>analyze_blood_report(image_url: str) -> dict</code> — the only tool this server currently exposes
Tool discovery	The client asking the server "what tools do you	Happens automatically when ChatGPT connects; FastMCP auto-generates the schema from the function's type hints and docstring — you never hand-write this schema

MCP concept	What it means	In this project
	have, and what do they need?"	
Tool invocation	The client actually calling a tool with real arguments	ChatGPT sending <code>analyze_blood_report(image_url="https://.../uploads/xyz.png")</code>
Transport	The underlying mechanism used to send MCP messages back and forth	Streamable HTTP — ordinary HTTP requests/responses, which is what lets this run as a normal web server reachable over the internet (as opposed to <code>stdio</code> transport, used for a <i>local</i> MCP server talking to a <i>local</i> AI client on the same machine — this project was tested with <code>stdio</code> early on via MCP Inspector, but the deployed version uses HTTP)
MCP resources / MCP prompts	Two other MCP concepts (exposing readable data, or reusable prompt templates)	Not used in this project — only a tool is defined

This project's actual MCP flow, replacing the protocol's general shape with what's really implemented here:



► Mermaid source for the diagram above

Where the actual MCP protocol implementation lives: entirely inside the `fastmcp` library (a dependency, not code in this repo). This project only uses FastMCP's decorator (`@mcp.tool`) and its ASGI app factory (`mcp_server_instance.http_app(...)`) — the message formats, request routing, and session handling are all implemented by that library, not hand-written here.

11. Application ↔ AI ↔ MCP ↔ External System

This section draws the line between four things that are easy to blur together: the app's own business logic, the AI's reasoning, MCP as a transport, and the actual external AI provider (OpenRouter).

```

Application (this repo's Python code)
┆ plain function calls, no protocol involved
AI / LLM (called via the openai SDK, hosted by OpenRouter)
┆ (separate concern – this app never itself acts as an MCP client)
MCP Client (ChatGPT – outside this repo entirely)
┆ MCP protocol over Streamable HTTP
MCP Server (mcp_server/server.py, inside this repo)
┆ plain function call
Application again (pipeline.analyze_report, same code as above)
┆
External System (OpenRouter, and the /uploads static file store)

```

The clearest way to describe this is: **this application is simultaneously an MCP server AND a user of AI models — but those are two separate, unrelated integrations.**

- **When this app calls an AI model** (Stage 1's vision call, Stage 3's embeddings, Stage 4's text generation), it is acting as a plain client of OpenRouter's API. There is **no MCP involved at all** in this direction. It's the same kind of call you'd make with any HTTP-based AI API — no tools, no protocol handshake, just "send a prompt, get text back."
- **When ChatGPT calls this app**, it is acting as an MCP client, and this app is acting as an MCP server. **The AI models this app itself calls (OpenRouter) are invisible to ChatGPT** — ChatGPT has no idea that, internally, `analyze_blood_report` is going to trigger three or four more separate AI calls to a completely different provider. From ChatGPT's point of view, it called one tool and got one structured result back.

Who initiates what: - A human initiates everything, ultimately — either by calling the REST API directly, or by typing a message to ChatGPT. - **ChatGPT (the LLM/agent) decides**, on its own, whether and when to call the `analyze_blood_report` tool — this app has no influence over that decision beyond writing a clear tool description (the docstring). - This app's own code decides, deterministically, whether a step needs an AI call at all (e.g. Stage 2 never does, regardless of what any LLM "wants").

Where business logic lives: entirely inside this repo's plain Python functions (`categorize()` , `generate_advice()` , `analyze_report()`). Neither OpenRouter's models nor ChatGPT ever make a decision that this app's own code depends on — the AI models only ever produce *content* (extracted text/JSON, or advice text), which this app's code then processes with its own rules.

Does the LLM ever access external data directly? No — not in either direction. The vision/text models called via OpenRouter never fetch anything themselves; they only ever see what's included directly in the prompt (the base64 image, or the retrieved knowledge-base text). ChatGPT never fetches the report image itself either — it only ever hands a URL string to the MCP tool, and this app's own server code (`httpx.get(...)`) is what actually performs that fetch.

What happens when a tool call fails: if `httpx.get()` inside the MCP tool raises (bad URL, network failure, non-2xx status via `raise_for_status()`), or if `analyze_report()` itself raises (e.g. every fallback AI model failed), that exception is not caught anywhere inside `mcp_server/server.py` — it propagates up through FastMCP's own request-handling code, which is responsible for turning it into an MCP-level error response back to the client. This app does not have any custom error-formatting logic specifically for the MCP path — compare this to the REST path (`main.py`), which *does* explicitly catch exceptions and turn them into a clean `HTTPException(500, ...)` with a message. This asymmetry is a real, current gap — noted in [Section 23](#).

12. Data Flow

```
Image file (user's blood report photo)
  ↓ read as bytes, base64-encoded
Prompt sent to a vision model (multimodal message)
  ↓ model responds with a JSON string
Python dict: {biomarker_name: numeric_value}
  ↓ compared against REFERENCE_RANGES
Python dict: {biomarker_name: {value, status, normal_range}}
  ↓ filtered to only High/Low entries
Search queries (one per abnormal biomarker, e.g. "Hemoglobin is Low")
  ↓ each embedded, compared against vector_store.json via cosine similarity
Retrieved knowledge-base paragraphs (top match per finding)
  ↓ assembled into one big prompt, with the findings summary
Prompt sent to a text model
  ↓ model responds with plain-language advice text
Final result: {"biomarkers": ..., "categorized": ..., "advice": ...}
  ↓ returned as JSON (REST) or as an MCP tool result
```

Key object shapes, exactly as they appear in the code:

- **Extracted biomarkers** (Stage 1 output): `{"Hemoglobin": 10.6, "Platelet Count": 128, ...}` — flat dict, name → number.
- **Categorized result** (Stage 2 output, and the final API's `categorized` field): `{"Hemoglobin": {"value": 10.6, "status": "Low", "normal_range": "13.0–17.0 g/dL"}}`.
- **Vector store entry** (one item inside `vector_store.json`'s list): `{"article": "anemia_low_hemoglobin.md", "chunk_index": 0, "text": "...", "embedding": [0.0123, -0.045, ...]}`.
- **Retrieved chunk** (what `retrieve_relevant_chunks` returns): `{"article": ..., "text": ..., "score": 0.6489}`.
- **Final pipeline result** (identical shape for both the REST response and the MCP tool result): `{"biomarkers": {...}, "categorized": {...}, "advice": "..."}.`
- **MCP tool argument**: a single string, `image_url`.
- **Upload endpoint response**: `{"url": "/uploads/<uuid>.png"}`.

13. Configuration & Environment

All configuration lives in a single `.env` file at the repo root (never committed — it's in `.gitignore`), loaded via `python-dotenv`'s `load_dotenv()`. Note this function is called independently in **three different files** (`pipeline.py`, `advisor.py`, `embedder.py`) rather than once centrally — each file that needs environment variables loads them itself.

Variable	Required?	Default if missing	Consumed in	What it controls
<code>OPENROUTER_API_KEY</code>	Yes, for any real AI call	none — calls will fail with an auth error if missing	<code>pipeline.py</code> , <code>advisor.py</code> , <code>embedder.py</code>	Your credential for OpenRouter. Every AI call in the project (vision, text generation, embeddings) is authenticated with this same key.

Variable	Required?	Default if missing	Consumed in	What it controls
MOCK_AI	No	"false"	pipeline.py only	When "true", skips every real AI call entirely and returns canned data from mock_data.py . Read once: os.getenv("MOCK_AI", "false").lower() == "true" .
BACKEND_API_URL	No	"http://127.0.0.1:8001"	mcp_server/server.py only	The base URL the MCP tool resolves a relative /uploads/... path against, before fetching the image. Should be set to wherever main.py 's server is actually reachable from (e.g. the ngrok public URL, in a remote-testing setup).

```
# .env – placeholders, not real values
OPENROUTER_API_KEY=<your-openrouter-key-here>
MOCK_AI=false
BACKEND_API_URL=http://127.0.0.1:8001
```

Other configuration, hardcoded directly in source (not environment-driven): - The three model IDs in VISION_MODELS (pipeline.py) and TEXT_MODELS (advisor.py) — identical lists in both files, defined separately rather than shared from one place. - The embedding model name, "nvidia/llama-nemotron-embed-v1-1b-v2:free", hardcoded in embedder.py. - OpenRouter 's base_url, "https://openrouter.ai/api/v1" — hardcoded identically in all three files that create an OpenAI client. - The port the app runs on (8001) — set on the command line when starting Uvicorn, not stored in any config file. - CORS settings in main.py — allow_origins=["*"] , wide open, hardcoded (flagged as a pre-deployment TODO in the README itself).

Authentication: the MCP server currently has **no authentication at all** — anyone who has the public ngrok URL can call the tool, or hit /upload / /analyze-report directly. This is explicitly a deliberate, temporary choice for testing (documented as "No Auth (current setup deliberately uses 'No Auth' for testing only)" in LEARNING_JOURNAL.md), not an oversight — but it's a real, current state worth knowing before ever pointing this at anything beyond local testing.

14. Dependencies

There is no requirements.txt , pyproject.toml , or Pipfile anywhere in this repo. The only place dependencies are listed at all is one pip install line in README.md . The table below is cross-checked against what's actually installed in the project's venv/ and what each package is actually imported for in the code.

Dependency	Purpose	Used in	Verified actually imported?
fastapi	Web framework — defines HTTP routes	main.py	Yes

Dependency	Purpose	Used in	Verified actually imported?
<code>uvicorn</code>	The actual server process that runs the FastAPI app	run from the command line, not imported in code	Yes (as a CLI tool)
<code>openai</code>	SDK for calling OpenRouter's OpenAI-compatible API	pipeline.py , advisor.py , embedder.py	Yes
<code>python-dotenv</code>	Loads <code>.env</code> into environment variables	same three files above	Yes (imported as <code>dotenv</code>)
<code>numpy</code>	Vector math (cosine similarity)	retriever.py	Yes
<code>fastmcp</code>	Builds the MCP tool server	mcp_server/server.py	Yes
<code>httpx</code>	HTTP client, used server-side to fetch an image by URL	mcp_server/server.py	Yes
<code>pillow</code>	Listed in the README's install command	—	No — not imported anywhere in the codebase. This appears to be a leftover from an earlier plan (perhaps for image validation/processing) that was never implemented, or a copy-paste artifact in the README. Flagged as a documentation inaccuracy.

None of these packages have pinned version numbers recorded anywhere in the repo — the exact versions currently installed (from the active `venv/`) are `fastapi 0.141.1`, `fastmcp 3.4.7`, `httpx 0.28.1`, `numpy 2.2.6`, `openai 3.2.0`, `python-dotenv 1.2.3`, `uvicorn 0.52.4`. Without a lock file, a fresh `pip install` on a new machine could pull different (potentially incompatible) versions in the future.

15. Error Handling

Failure scenario	What actually happens	Where in the code
Uploaded file isn't an image (<code>/analyze-report</code> or <code>/upload</code>)	Immediate <code>HTTPException(400, "...")</code> , checked via <code>file.content_type.startswith("image/")</code> — note this trusts a client-supplied header, it does not inspect the actual file bytes	main.py
A vision model call fails or returns bad JSON	Caught by a broad <code>except Exception</code> , recorded as <code>last_error</code> , next attempt/model is tried. If every model/attempt in <code>VISION_MODELS</code> fails, the last exception is re-raised	<code>extract_biomarkers()</code> in pipeline.py
A text-generation call fails	Same pattern, but with a real sleep-based backoff (<code>attempt * 5</code> seconds) between retries, since these failures are more often genuine rate-limit congestion	<code>call_model_with_fallback()</code> in advisor.py
An embedding call fails	Its own independent retry loop, 3 attempts, same backoff style; re-raises on the final failure	<code>embed_text()</code> in embedder.py
The pipeline raises for any reason, via the REST	Caught in <code>main.py</code> 's <code>try/except Exception</code> as <code>e:</code> , turned into <code>HTTPException(500, detail=f"Analysis</code>	main.py

Failure scenario	What actually happens	Where in the code
route	<code>failed: {str(e)}</code> — the temp file is still deleted via <code>finally</code> regardless	
The pipeline raises for any reason, via the MCP tool	Not caught anywhere in <code>mcp_server/server.py</code> . The exception propagates up into FastMCP's own request handling, which is responsible for converting it into an MCP-level error the calling AI client sees. There's no custom, human-readable error message crafted for this path the way there is for the REST path — a real asymmetry between the two entry points (see Section 23)	mcp_server/server.py
The MCP tool is given a URL that doesn't point to a real image (bad URL, 404, non-image content)	<code>httpx.get(...).raise_for_status()</code> will raise on a non-2xx response and stop immediately. If the URL <i>does</i> return 200 but the bytes aren't actually a valid image, there's no check for this — the bytes get written to a <code>.png</code> file regardless, and the failure only surfaces later, inside Stage 1, when the vision model can't make sense of it	mcp_server/server.py
<code>OPENROUTER_API_KEY</code> missing/invalid	Not checked proactively anywhere — the first real AI call will simply fail with an authentication error from OpenRouter, which then flows through the same retry/fallback/500 path as any other AI failure	implicit, via any OpenRouter call

16. Logging & Debugging

There is **no logging framework** used anywhere in this codebase — every diagnostic message is a plain `print(...)` statement (e.g. `print(f"Trying model: {model} (attempt {attempt})...")`). This means: no log levels (info/warning/error), no structured/JSON logs, and no way to filter verbosity — everything printed goes straight to whatever terminal is running `uvicorn`. This is explicitly called out as a known gap in the project's own roadmap ("Structured logging in place of `print()` statements").

A practical way to debug an issue in this app, adapted to how it's actually built:

```

Something went wrong
↓
Check the terminal running `uvicorn main:app` — every retry/fallback attempt
prints a line (e.g. "Trying model: X (attempt N)..." / "Failed (...)").
↓
Was it a REST call? Check for the HTTPException status/detail in the response —
main.py always includes the real exception message in a 500's "detail" field.
↓
Was it an MCP/ChatGPT call? There's no equivalent friendly message — check the
raw server logs directly for the real Python exception, since it isn't
reformatted before reaching the AI client.
↓
Is MOCK_AI accidentally "true"? That would explain a result that looks static
or ignores the actual uploaded image.
↓
Is this a "used to work" regression after a restart? Check `which python3` —
the recurring root cause throughout this project's history has been a
deactivated venv in a fresh terminal (see LEARNING_JOURNAL.md — hit at least
three separate times).
↓
Is it specifically a ChatGPT/MCP mismatch (wrong arguments sent)? ChatGPT can
cache an old tool schema after the tool's parameters change — removing and
re-adding the connector (not just toggling it) forces a fresh schema fetch.

```

17. Testing

There are no automated tests anywhere in this repository — no `pytest`, no test files, no CI configuration. This is confirmed by the repo's file listing (no `test_*.py` or `tests/` directory exists) and matches the project's own roadmap, which lists "Proper testing (pytest)" as not yet started.

What currently substitutes for testing: - **playground.py** — a manual scratch script for poking at retrieval behavior. - **pipeline.py's** `if __name__ == "__main__": block` — running `python3 pipeline.py` is a manual, human-observed smoke test of the whole pipeline against `sample_report.png`. - **MOCK_AI=true** — lets a human manually exercise the entire REST/MCP plumbing (uploads, routing, response shape) with zero real network calls, though this only proves the *plumbing* works, not that the AI logic itself produces correct results. - **MCP Inspector** (`npx @modelcontextprotocol/inspector ...`) — a manual, browser-based tool for triggering the MCP tool directly, used during development of `mcp_server/server.py`. - **The actual ChatGPT connector** — the most "real" end-to-end validation performed so far, done manually and once (documented in the README's MCP section), not something that runs repeatably.

Not covered by anything today: automated verification that `categorize()` produces correct High/Low/Normal tags, that `chunk_text()` splits correctly at edge cases (e.g. text shorter than one chunk), that the fallback loops actually move to the next model after real failures, or that the API's error responses have the right shape.

18. Running the Application

These are the exact, verified commands — matching what the code and `.env` actually expect (port `8001`, not the `8000 / 8005` you may see in older references).

1. Set up the environment

```
python3 -m venv venv
source venv/bin/activate      # on Windows: venv\Scripts\activate
pip install fastapi uvicorn openai python-dotenv numpy fastmcp httpx
```

(`pillow`, listed in the README, is not actually required by any code — see [Section 14.](#))

2. Configure `.env` (repo root)

```
OPENROUTER_API_KEY=your-openrouter-key-here
MOCK_AI=true
BACKEND_API_URL=http://127.0.0.1:8001
```

Start with `MOCK_AI=true` for your first run — it costs nothing and proves the plumbing works before spending any real AI quota.

3. Build the knowledge base (only needed once, or after editing `articles/`)

```
python3 build_vector_store.py
```

4. Start the server

```
uvicorn main:app --reload --port 8001
```

5. Verify it's running

```
curl http://127.0.0.1:8001/  
# {"status": "Blood Report Analyser API is running"}
```

6. Try the real endpoint

```
curl -X POST http://127.0.0.1:8001/analyze-report -F "file=@sample_report.png"
```

7. (Optional) Test the CLI pipeline directly, without the server

```
python3 pipeline.py
```

8. (Optional) Expose it publicly for an AI assistant like ChatGPT

```
ngrok http 8001
```

Register `https://<ngrok-subdomain>.ngrok-free.dev/mcp` as the MCP connector URL in ChatGPT's connector settings. Remember: this free ngrok URL changes every time the tunnel restarts, and (per [Section 13](#)) there is currently no authentication on this endpoint at all while it's exposed.

19. Complete Practical Example

This traces one real, already-verified request end to end (the exact scenario documented in the README, run in `MOCK_AI=true` mode).

```

User has a blood report photo, wants ChatGPT to analyze it
↓
[B1] curl -X POST https://<ngrok>/upload -F "file=@report.png"
    → main.py: upload_report() saves it as uploads/0cd76a6d-....png
    → returns {"url": "/uploads/0cd76a6d-....png"}
↓
[B2] User, in ChatGPT: "Analyze this blood report using the Blood Report
    Analyser tool: https://<ngrok>/uploads/0cd76a6d-....png"
↓
[B3] ChatGPT (its own reasoning, outside this repo) decides to call:
    analyze_blood_report(image_url="https://<ngrok>/uploads/0cd76a6d-....png")
↓
[B4] Request arrives at .../mcp on the ngrok tunnel → forwarded to
    localhost:8001/mcp → routed by FastMCP to analyze_blood_report()
↓
[B5] mcp_server/server.py: httpx.get(...) fetches the image bytes itself,
    writes them to a temp .png file
↓
[B6] pipeline.analyze_report(tmp_path) runs:
    - Stage 1 (MOCK_AI=true): returns MOCK_BIOMARKERS immediately, no AI call
    - Stage 2: categorize() flags Hemoglobin 10.6 → Low, Hematocrit 38.2% →
      Low, Total Leukocyte Count 13.8 → High, Platelet Count 128 → Low
      (compared against reference_ranges.py's min/max table)
    - Stage 3+4 (MOCK_AI=true): returns MOCK_ADVICE immediately, no AI call
↓
[B7] Result: {"biomarkers": {...}, "categorized": {...}, "advice": "This is
    placeholder advice generated in MOCK_AI mode..."} returned as the tool
    result, back through /mcp, through ngrok, to ChatGPT
↓
[B8] ChatGPT renders this as a conversational answer: a table of the four
    abnormal results with their reference ranges and status, a bulleted
    list of normal values, a "what this may indicate" explanation (anemia
    pattern given normal MCV, elevated WBC possibly infection/inflammation/
    stress, mild thrombocytopenia), and a closing recommendation to discuss
    with a doctor – correctly noting the data came from mock/test mode.

```

Every file/function named above is exactly what runs — nothing in this example is hypothetical.

20. How to Read This Codebase

Recommended order for a new developer, and why:

1. README.md – the project's own framing of what it is and how to run it
2. LEARNING_JOURNAL.md – the "why," in the original author's own words, including every real bug hit and how it was diagnosed
3. main.py – the actual entry point; see where routes are defined and how the MCP server gets mounted in
4. pipeline.py – the single function that ties every stage together; read this before any individual stage file
5. categorize.py + reference_ranges.py – the simplest stage; a good warm-up before the AI-heavy stages
6. advisor.py + retriever.py + embedder.py – the RAG stages, in the order data actually flows through them
7. chunker.py + load_articles.py + build_vector_store.py – the offline prep step that produces vector_store.json
8. mcp_server/server.py – now that you know what analyze_report() does, see how it's exposed as an AI-callable tool
9. mock_data.py – small, but explains what "MOCK_AI=true" actually changes at each stage
10. This document (LEARNING.md) – cross-check your understanding against it

Why this order: it follows the same path a real request actually takes (entry point → orchestrator → each stage in execution order → the offline prep behind Stage 3 → the second entry point that reuses everything). Reading `mcp_server/server.py` before `pipeline.py` would leave you looking at `analyze_report(tmp_path)` with no idea what it does yet.

21. Learning Journey & Architectural Decisions

This section connects specific entries in `LEARNING_JOURNAL.md` to exactly where their outcome lives in the code today.

Decision: rule-based code instead of AI for categorization - *Why it was needed:* every extracted biomarker value needs a High/Low/Normal tag. - *Options considered:* ask an AI model to make the call, vs. plain code comparing against a table. - *Chosen approach:* plain code — `categorize.py` + `reference_ranges.py`. - *Why:* comparing a number to a range has exactly one correct answer; using an AI model for a deterministic task adds cost, latency, and needless risk of error with zero upside. - *Trade-off:* the reference table must be manually maintained and only covers biomarkers someone explicitly added; anything else is tagged "Unknown." - *Where implemented:* [categorize.py](#), [reference_ranges.py](#).

Decision: a flat JSON file instead of a real vector database - *Why it was needed:* Stage 3 needs somewhere to store and search article-chunk embeddings. - *Options considered:* a managed vector database (Pinecone, Chroma, etc.) vs. a plain JSON file plus manual cosine similarity. - *Chosen approach:* the JSON file (`vector_store.json`) + `retriever.py`'s from-scratch similarity search. - *Why:* this is a learning project — the goal was to see the retrieval mechanics directly (embed, compare, sort, return top-k) instead of having a library hide them. - *Trade-off:* a full linear scan over every chunk on every query; fine at ~50 chunks, would not scale to a large knowledge base. - *Where implemented:* [retriever.py](#), generated by [build_vector_store.py](#).

Decision: image_url instead of image_base64 for the MCP tool - *Why it was needed:* the original `image_base64: str` tool signature caused ChatGPT's client to hang indefinitely while base64-encoding an 83KB image — it never even reached the server. - *Options considered:* MCP's native `ImageContent` type (rejected — documented as not rendering correctly in ChatGPT at the time); a `file://` local path (rejected — assumes client and server share a filesystem, which is false for a remote/cloud AI host reaching a server over ngrok). - *Chosen approach:* a new `POST /upload` endpoint returns a URL; the tool takes that URL and fetches the bytes itself, server-side, via `httpx`. - *Why:* no binary data ever has to pass through an MCP tool argument at all — only a short string does. - *Where implemented:* `main.py`'s `/upload` route, `mcp_server/server.py`'s `httpx.get(...)`.

Decision: merge the MCP server into the main FastAPI backend - *Why it was needed:* once the MCP tool needed to fetch images from a *publicly reachable* URL (ChatGPT correctly refuses to fetch `127.0.0.1` addresses), both the backend and the MCP server needed to be tunneled — but ngrok's free tier only allows **one active tunnel at a time** (`ERR_NGROK_334` was hit when a second was attempted). - *Options considered:* pay for a second ngrok tunnel, vs. combine both services into one process/port. - *Chosen approach:* mount the FastMCP app inside the FastAPI app (`app.mount("/mcp", mcp_app)`), sharing one Uvicorn process and one ngrok tunnel. - *Trade-off:* the two services are now coupled to the same deploy/restart lifecycle — you can no longer restart just the MCP layer independently. This is also **why** `mcp_server/server.py`'s old standalone `if __name__ == "__main__":` entry point became dead code and was subsequently removed (see 6.2). - *Where implemented:* [main.py](#).

Decision: MOCK_AI flag for cost-free development - *Why it was needed:* real AI calls cost quota (OpenRouter's free tier caps at 50 requests/day per account, shared across every model), and iterating on plumbing (uploads, routing, response shape) doesn't need real AI output every time. - *Chosen approach:* a single environment variable, checked at the top of `extract_biomarkers()` and `analyze_report()`, swapping in `mock_data.py`'s canned values. - *Where implemented:* [pipeline.py](#), [mock_data.py](#).

Observation (not a documented decision, but visible directly in the code): three separate OpenAI client instances - `pipeline.py`, `advisor.py`, and `embedder.py` each independently create their own `OpenAI(base_url=..., api_key=...)` client and call `load_dotenv()` themselves, rather than sharing one client/config module. This isn't called out as a deliberate decision anywhere in the journal — it reads as organic growth (each file was built and tested somewhat independently) rather than an intentional architectural choice. See [Section 23](#).

22. FAQ — Common Questions a New Developer Will Have

Where does the application actually start? Running `uvicorn main:app` starts everything — `main.py` is the true entry point. There's a second, independent entry point for manual testing: `python3 pipeline.py`.

Which function is "the" entry point into the real logic? `analyze_report(image_path)` in `pipeline.py` — both the REST route and the MCP tool call this exact function.

Where is the AI model actually called? Three places: `extract_biomarkers()` in `pipeline.py` (vision), `call_model_with_fallback()` in `advisor.py` (text generation), and `embed_text()` in `embedder.py` (embeddings). All three go through the `openai` SDK pointed at OpenRouter.

Where is the prompt defined, and where would I change it? The extraction prompt is the `prompt_text` string inside `extract_biomarkers()` in `pipeline.py`. The advice-generation prompt is built inside `generate_advice()` in `advisor.py`.

Where would I change which AI model is used? Edit the `VISION_MODELS` list in `pipeline.py` or the `TEXT_MODELS` list in `advisor.py`. The embedding model name is a single hardcoded string inside `embed_text()` in `embedder.py` — note that changing it would require re-running `build_vector_store.py`, since the query embedding and the stored article embeddings must come from the same model to be comparable.

Where is MCP configured / where is the MCP server? `mcp_server/server.py` defines the tool; `main.py` mounts it into the running app at `/mcp`.

How does the LLM (ChatGPT) decide to use the tool? Entirely on its own, based on the tool's name, parameter types, and docstring (auto-turned into its description by FastMCP) — this app has no code that influences that decision beyond writing a clear docstring.

How does data reach the MCP server if the client is ChatGPT, running in the cloud? Through a public ngrok tunnel pointed at the local server's port — ChatGPT's request goes to the public ngrok address, ngrok forwards it to `localhost:8001`.

Where can I add a new MCP tool? In `mcp_server/server.py`, define a new function and decorate it with `@mcp.tool`, following the pattern of `analyze_blood_report`. (The project's own roadmap already envisions `explain_biomarker`, `get_reference_range`, and `generate_report_pdf` as future tools — none exist in the code yet.)

Where can I add a new REST API route? In `main.py`, following the pattern of the existing `@app.get(...)` / `@app.post(...)` routes.

Where should new business logic go? Inside `pipeline.py` or a new dedicated module imported by it — keep it out of `main.py` / `mcp_server/server.py`, which are meant to stay thin "entry point" wrappers around the pipeline.

How do I debug a failed AI request? Check the terminal running Uvicorn — every retry attempt prints its own line. See [Section 16](#).

How do I debug a failed MCP request specifically? Same terminal, same approach — but remember MCP-path failures aren't reformatted into a friendly message the way REST failures are, so look for the raw Python traceback.

How do I run the tests? There currently are none to run — see [Section 17](#).

How do I add a new feature, generally? Given the "single source of truth" pattern, ask: does this belong in the shared pipeline (`pipeline.py` / `advisor.py` / `categorize.py` , so both REST and MCP get it automatically), or is it specific to just one entry point (a route-only concern in `main.py` , or a tool-only concern in `mcp_server/server.py`)?

23. Code Quality: Strengths, Risks & Improvement Opportunities

Everything in this section is an observation/recommendation, not a description of a problem that has been fixed. No code was changed as part of writing this document.

Current strengths

- **One pipeline, two entry points, zero duplication** — both the REST route and the MCP tool call the exact same `analyze_report()` function. A fix or improvement to the pipeline logic benefits both automatically.
- **Deliberate, explained use of AI vs. plain code** — Stage 2 is intentionally *not* AI, and this reasoning is documented, not just implicit.
- **Real resilience patterns already in place** for AI calls: retries, backoff, multi-model fallback, and a defensive check for empty/malformed model responses.
- **Mock mode** genuinely allows zero-cost, zero-network testing of the full plumbing.
- **The MCP image-handling redesign** (base64 → URL fetch) reflects a real, correctly-diagnosed fix to a genuine cross-client limitation, not a workaround for a self-inflicted bug.

Risks & technical debt

- **No authentication on the MCP server or the REST API.** While tunneled via ngrok, `/analyze-report` , `/upload` , and the MCP tool are all reachable by anyone with the URL. `/upload` in particular allows arbitrary file uploads into `uploads/` with no size limit and only a client-supplied `Content-Type` check (easily spoofed) — this is both an abuse vector (storage exhaustion) and a soft validation gap.
- **The MCP tool performs a server-side `httpx.get()` on any URL an AI client hands it**, with no restriction on target host. If this server is ever deployed somewhere with access to internal/private network addresses, this is a textbook SSRF (server-side request forgery) shape: the AI (or anything able to influence the tool argument) could ask the server to fetch an internal-only URL it otherwise wouldn't have access to.
- **Inconsistent error handling between the two entry points.** The REST path wraps pipeline failures into a clean `HTTPException(500, ...)` ; the MCP path has no equivalent — an unhandled exception propagates directly into FastMCP's own error handling with no app-specific message.
- **No dependency manifest** (`requirements.txt` / `pyproject.toml`) and no pinned versions — a new environment could install different package versions than the ones this was built/tested against.
- **Documentation drift:** the README's install instructions list `pillow` , which no code in the repo actually imports.
- **Duplicated configuration across files** — three separate `OpenAI` client instantiations (same `base_url` , same API key source) and three separate `load_dotenv()` calls, instead of one shared client/config module.
- **`vector_store.json` is reloaded from disk on every single `/analyze-report` request** (via `load_vector_store()` inside `analyze_report()`) — for a ~2MB file, this is a real, repeated cost with no

caching across requests within the same running server process.

- **No automated tests at all.**
- **Only `print()` for logging** — no log levels, no way to control verbosity, no structured/searchable logs.

AI/LLM-specific considerations

- Structured output relies entirely on prompt wording plus `json.loads()`, with no native JSON-mode/function-calling enforcement from the provider — the retry/fallback logic exists specifically to paper over this. Some OpenRouter models do support stricter structured-output modes; none are currently used.
- Free-tier models mean the exact model IDs in `VISION_MODELS` / `TEXT_MODELS` can become unavailable at any time (the project's own history includes exactly this happening) — there's no automated way to detect a "this model ID no longer exists" failure differently from an ordinary transient failure.
- The account-wide daily quota (50 free requests/day, shared across all models) is a distinct failure mode from per-model rate limiting, and the current fallback logic does nothing to help once that ceiling is hit — the project's own journal identifies this gap explicitly.

MCP-specific considerations

- Only one tool exists today (`analyze_blood_report`); no MCP **resources** or **prompts** are implemented, even though the protocol supports them.
- Tool-schema caching on the client side (observed with ChatGPT) means changing a tool's parameters requires the connector to be fully removed and re-added — there's nothing this server can do about that; it's a client-side behavior to be aware of when iterating on a tool's signature.

Security considerations (summary)

Wide-open CORS (`allow_origins=["*"]`), no MCP/API authentication, unrestricted server-side URL fetching in the MCP tool, and client-supplied (spoofable) `Content-Type` validation are all currently acceptable **only** because this is explicitly a local/tunneled learning project, not a deployed production service. Every one of these would need to be addressed before any real-world exposure — several are already tracked in the README's own roadmap (CORS restriction, real authentication).

Performance considerations (summary)

The vector store's linear-scan search and its full re-read from disk on every request are both fine at the current scale (a few dozen chunks) and would need revisiting only if the knowledge base or request volume grew meaningfully.

24. Glossary

Term	Simple explanation	Used in this project for
API	A defined way for two pieces of software to talk to each other, usually over HTTP	The whole <code>/analyze-report</code> , <code>/upload</code> REST interface in <code>main.py</code>
HTTP	The protocol web requests/responses are sent over	Every interaction with this app, including MCP's transport
JSON	A text format for structured data (objects/lists/numbers/strings)	Every request/response body, and the vision model's required output format

Term	Simple explanation	Used in this project for
LLM	A large language model — an AI that generates/understands text	The vision and text models called via OpenRouter
Vision-language model (VLM)	An LLM that can also read images	Stage 1's biomarker extraction
Prompt	The instruction text sent to an AI model	<code>prompt_text</code> in <code>pipeline.py</code> , the prompt built in <code>advisor.py</code>
Embedding	Text converted into a list of numbers representing its meaning	<code>embedder.py</code> , used for both articles and search queries
Vector store	Where embeddings are saved for later searching	<code>vector_store.json</code>
Cosine similarity	A math formula measuring how similar two embeddings' "directions" are	<code>retriever.py</code> 's core comparison
RAG (Retrieval-Augmented Generation)	Search your own data first, then have the AI answer using only what was found	Stages 3+4 together
Grounding	Restricting an AI's answer to only the information it was explicitly given	The "Using ONLY the reference information above" instruction in <code>advisor.py</code>
Hallucination	An AI stating something false/unsupported as if it were fact	The exact failure mode grounding is meant to prevent
MCP (Model Context Protocol)	A standard way for an AI application to call external tools/data	<code>mcp_server/server.py</code> , mounted into <code>main.py</code>
MCP server	The program exposing callable tools	<code>mcp_server/server.py</code>
MCP client	The AI application connecting to an MCP server	ChatGPT, in the tested integration
MCP tool	One specific callable function exposed to an AI	<code>analyze_blood_report</code>
Tool discovery	The client asking "what tools/parameters do you have?"	Handled automatically by FastMCP from type hints + docstring
Transport	The underlying mechanism messages travel over	Streamable HTTP, in this project's deployed setup
ASGI app	The standard interface Python async web servers/frameworks use	How the FastMCP tool server is mounted inside FastAPI
Decorator	A function that wraps another function to add behavior	<code>@app.get(...)</code> , <code>@mcp.tool</code>
Context manager (<code>with</code>)	A pattern guaranteeing cleanup code runs automatically	Every <code>with open(...)/with tempfile...</code> in this codebase
Environment variable	Configuration passed to a program from outside its code	<code>OPENROUTER_API_KEY</code> , <code>MOCK_AI</code> , <code>BACKEND_API_URL</code>
Virtual environment (<code>venv</code>)	An isolated, project-specific copy of Python + its packages	The <code>venv/</code> folder; must be activated per terminal session
ngrok	A tool that gives a local server a temporary public internet address	Used to let ChatGPT reach the locally-run server
Mock mode	Swapping real, costly operations for fixed fake data during testing	<code>MOCK_AI=true</code> , <code>mock_data.py</code>

25. Quick Reference

I want to...	Go to...
Start the server	<code>uvicorn main:app --reload --port 8001</code>
Run the pipeline without a server	<code>python3 pipeline.py</code>
Test with zero AI cost	Set <code>MOCK_AI=true</code> in <code>.env</code>
Rebuild the knowledge base after editing an article	<code>python3 build_vector_store.py</code>
Change which AI models are tried	<code>VISION_MODELS</code> in pipeline.py , <code>TEXT_MODELS</code> in advisor.py
Change the extraction prompt	<code>prompt_text</code> inside <code>extract_biomarkers()</code> in pipeline.py
Change the advice-generation prompt	Inside <code>generate_advice()</code> in advisor.py
Add/adjust a biomarker's normal range	reference_ranges.py
Add a new knowledge-base article	Add a <code>.md</code> file to <code>articles/</code> , then rebuild the vector store
Add a new MCP tool	mcp_server/server.py , following the <code>@mcp.tool</code> pattern
Add a new REST route	main.py
Expose the app to ChatGPT for testing	<code>ngrok http 8001</code> , register <code>.../mcp</code> as the connector URL
Trace a request end to end	Section 7